Correctness and Termination Proofs

Learning Outcomes

by the **End** of the **Lecture**, **Students** that **Complete** the **Recommended Exercises** should be **Able** to:

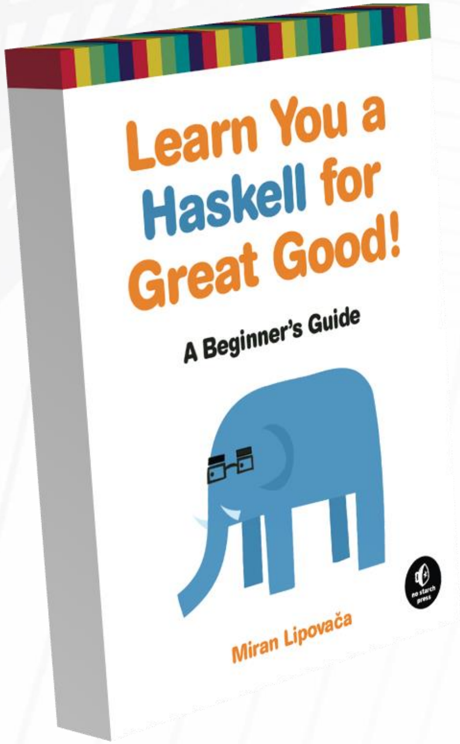
Contrast Proof with White-Box and Black-Box Testing

Contrast Partial and Total Correctness

Describe Termination and the use of Rank Functions

Describe Structural Induction

Reading and Suggested Exercises



Read the following **Chapter(s)**:

None

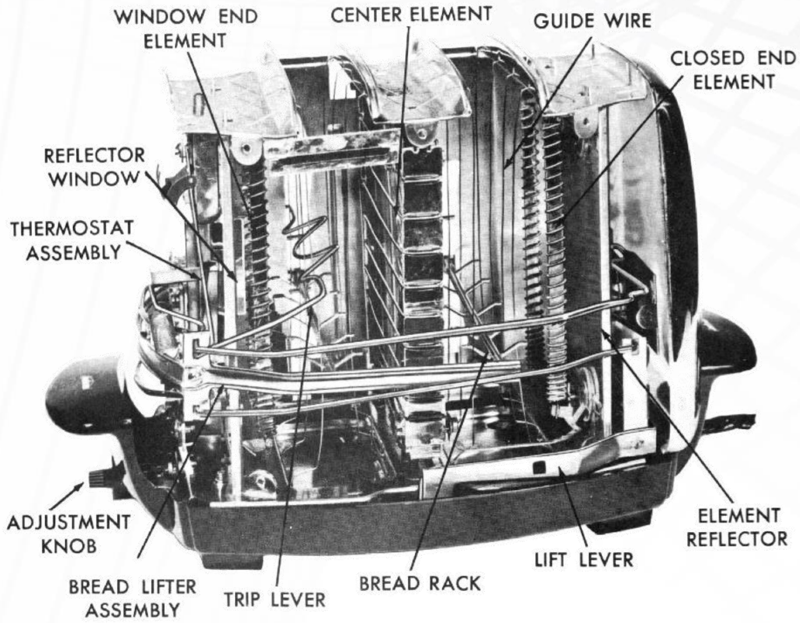
Alan Turing proved that it is **Impossible** to **Create** an **Algorithm** that will **Determine** if a **Program** will **Terminate** on a **Certain Input** (this is known as the "**Halting Problem**")

Testing provides **Confidence** in the **Correctness** of a **Program** and testing **Techniques** can be **Divided** into **Black-Box** and **White-Box Testing**

White-Box Testing is **Characterized** by **Examining Internal Structure** and **Testing** each **Logical Case**

this **Approach** is frequently **Infeasible**
(e.g., $\boldsymbol{n}$ **Branching Statements** entails $\mathbf{2^n}$ **Combinations**)

White-Box Testing is **Analogous** to using **Perfect Information** about a **Device** to **Ensure** that **Every Possibility** is **Tested**



Black-Box Testing is **Characterized** by **Testing Programs Without Observing Internal Structure**

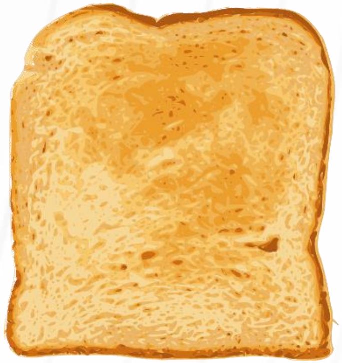
this **Approach Compares** the **Results** on **Problem Instances** for which the **Correct Result** is **Known**

Program Testing

Black-Box Testing is **Analogous** to **Testing** a **Device** by **Using** it, and **Entails Testing Three Input Classes**: "**Good**", "**Bad**", and "**Ugly**"

Program Testing

Black-Box Testing is **Analogous** to **Testing** a **Device** by **Using** it, and **Entails Testing Three Input Classes**: "**Good**", "**Bad**", and "**Ugly**"



the "**Good**"

the "**Bad**"

the "**Ugly**"

Program Testing

I have included this image in case you thought the analogy of someone trying to toast a croissant in an automatic toaster was too contrived...

https://www.wikihow.com/Heat-Croissants

Testing Can Indicate the **Presence** of **Problems**,
but it **Cannot Demonstrate** the **Absence** of **Problems**

Proofs of **Correctness**, on the other hand, are
Formal Proofs that **Programs** are **Free** of **Problems**

to **Prove** that a **Function** is **Totally Correct**
you must **Prove** that:

the **Function Terminates** whenever the **Input** is **Valid**
this is known as "**Termination**"

If the **Function Terminates**, **Then** the **Result** is **Correct**
this is known as "**Partial Correctness**"

Can you **Guarantee** that this **Function Terminates**?

Why or **Why Not**?

```
fact :: Integer -> Integer

fact n
  | (n == 0) = 1
  | otherwise = n * (fact (n-1))
```

to **Prove** that **Recursive Function** f **Terminates**, **Devise** a **Rank Function** that **Maps Each Input** onto a **Nonnegative Integer**…

…and then **Prove** that:

If function $\boldsymbol{f}$ is passed an **Argument** of **Rank** k **Then Rank** k must be **Greater Than** the **Rank** of the **Argument** passed to **Each Subsequent Recursive Call**

How could you **Determine** a **Suitable Rank Function** for `zip`?

```
zip :: [a] -> [b] -> [(a, b)]
zip (h1:t1) (h2:t2) = (h1,h2) : zip (t1 t2)
zip _ _ = []
```

n.b., each input to `zip` is two list arguments, so `rank :: [a] -> [b] -> Integer`

What are the **Arguments** that will be **Passed** to **Each Call** to zip in the **Following Evaluation**?

```
zip [1, 3, 25] ['a', 'c', 'y', 'z']
```

Formal Proofs of Termination

`zip [1, 3, 25] ['a', 'c', 'y', 'z']`

will make the **Recursive Call**

`zip [3, 25] ['c', 'y', 'z']`

will make the **Recursive Call**

`zip [25] ['y', 'z']`

will make the **Recursive Call**

`zip [] ['z']`

Now Suppose

`rank [1, 3, 25] ['a', 'c', 'y', 'z'] =` $\boldsymbol{rank_1}$

`rank [3, 25] ['c', 'y', 'z'] =` $\boldsymbol{rank_2}$

`rank [25] ['y', 'z'] =` $\boldsymbol{rank_3}$

`rank [] ['z'] =` $\boldsymbol{rank_4}$

What Function could you use that would **Guarantee**
$\boldsymbol{rank_1 > rank_2 > rank_3 > rank_4}$?

Consider the **Function Definition**

```
rank list1 list2 = min (length list1) (length list2)

rank [1, 3, 25] ['a', 'c', 'y', 'z']        = 3
    rank [3, 25] ['c', 'y', 'z']            = 2
        rank [25] ['y', 'z']                = 1
            rank [] ['z']                   = 0
```

Can you **Prove** this `rank` **Function Definition** will work in the **General Case**?

```
zip (h1:t1) (h2:t2) = (h1,h2) : zip (t1 t2)
```

What is the Rank of the **Arguments**?

What is the Rank of the **Recursive Arguments**?

```
zip (h1:t1) (h2:t2) = (h1,h2) : zip (t1 t2)
```

What is the Rank of the **Arguments**?

```
min(length(h1 : t1), length(h2 : t2))
    = min(1 + length(t1), 1 + length(t2))
```

What is the Rank of the **Recursive Arguments**?

```
min(length(t1), length(t2))
```

How do you **Prove** `min(1 + length(t1), 1 + length(t2))` is **Greater** than `min(length(t1), length(t2))`?

n.b., you can use a proof by exhaustion (i.e., proof by cases) to accomplish this easily

Prove `min(1 + length(t1), 1 + length(t2))`

is **Greater** than `min(length(t1), length(t2))`

Case 1: `length(t1) > length(t2)`

```
min(1 + length(t1), 1 + length(t2))
= 1 + length(t2)
> length(t2)
= min(length(t1), length(t2))
```

Prove `min(1 + length(t1), 1 + length(t2))`
is **Greater** than `min(length(t1), length(t2))`

Case 2: `length(t1)` $\leq$ `length(t2)`

```
min(1 + length(t1), 1 + length(t2))
      = 1 + length(t1)
        > length(t1)
  = min(length(t1), length(t2))
```

a **Function** that **Oberves** the **Constraints** of **Primitive Recursive** will always have **Termination**

it follows that, in order to **Demonstrate** that **Primitive Recursive Function** $\boldsymbol{f}$ is **Totally Correct**, you **Need** only **Demonstrate** that $\boldsymbol{f}$ is **Partially Correct**

i.e., "**If** f **Terminates Then** the **Result** was **Correct**"

Demonstrating Partial Correctness is the **Proof** of an **Implication Statement**, so typically **Structural Induction** is **Necessary**

Structural Induction requires **Assumptions** that **Lists** are of **Finite Length** and that **Functions** are **Defined** over all **Possible Inputs**

to **Prove** that a **Property** P holds for
All Finite Lists using **Structural Induction**:

Prove the **Property Holds** for the **Base Case**
i.e., $P(\,[\,]\,) = True$

Then Prove that, **Under** the **Assumption** $P(\,x\,) = True$,
the **Claim** $P(\,y : x\,) = True$ **Holds** as well

the **Assumption** is the **Induction Hypothesis**

How would you **Write** a **Function** (or functions) that **Computes** the **Sum** of the **Doubling** of a **List**?

```
doubleList :: [Int] -> [Int]

sumList :: [Int] -> Int
```

Formal Proofs of Correctness

```
doubleList :: [Int] -> [Int]

doubleList [] = []                                  D1
doubleList (h:t) = 2*h : doubleList t               D2
```

```
sumList :: [Int] -> Int

sumList [] = 0                                      S1
sumList (h:t) = h + sumList t                       S2
```

How can you **Prove** (using **Structural Induction**) that `sumList (doubleList x)` **Correctly Computes** the **Sum** of the **Doubling** of a **List**?

also, **What** is the **Rank Function**?

and, **Do** we **Need** such a **Function** to **Prove Total Correctness** here?

the **Conjecture** to be **Proven** here is that the **Result** is the **Same** as **Twice** the **Sum** of the **List**

i.e., **Proving** that this **Works** is (here) the **Same As Proving** `sumList(doubleList x) = 2 * sumList x` to **Prove** this **Using Structural Induction**...

first **Prove** the **Base Case**…

```
sumList(doubleList []) = 2 * sumList []
```

...then **Assume**…

```
sumList(doubleList t) = 2 * sumList t
```

...in order **To Prove**...

```
sumList(doubleList h:t) = 2 * sumList h:t
```

Formal Proofs of Correctness

Knowledge Base	
S_1	`sumList [] = 0`
S_2	`sumList (h:t) = h + sumList t`
D_1	`doubleList [] = []`
D_2	`doubleList (h:t) = 2*h : doubleList t`

Prove: `sumList(doubleList []) = 2 * sumList []`

Formal Proofs of Correctness

Knowledge Base	
S_1	`sumList [] = 0`
S_2	`sumList (h:t) = h + sumList t`
D_1	`doubleList [] = []`
D_2	`doubleList (h:t) = 2*h : doubleList t`

Prove: `sumList(doubleList []) = 2 * sumList []`

```
sumList([]) = 2 * sumList []        by D1
          0 = 2 * 0                 by S1
          0 = 0
Q.E.D.
```

Formal Proofs of Correctness

Knowledge Base	
S_1	`sumList [] = 0`
S_2	`sumList (h:t) = h + sumList t`
D_1	`doubleList [] = []`
D_2	`doubleList (h:t) = 2*h : doubleList t`
IA	`sumList(doubleList t) = 2 * sumList t`

Prove: `sumList(doubleList h:t) = 2 * sumList h:t`

Formal Proofs of Correctness

Knowledge Base	
S_1	`sumList [] = 0`
S_2	`sumList (h:t) = h + sumList t`
D_1	`doubleList [] = []`
D_2	`doubleList (h:t) = 2*h : doubleList t`
IA	`sumList(doubleList t) = 2 * sumList t`

Prove: `sumList(doubleList h:t) = 2 * sumList h:t`

```
sumList(2*h : doubleList t) = 2 * sumList h:t        by D2
2*h + sumList (doubleList t) = 2 * sumList h:t       by S2
2*h + (2 * sumList t) = 2 * sumList h:t              by IA
2*h + (2 * sumList t) = 2 * (h + sumList t)          by S2
2*h + (2 * sumList t) = 2*h + (2 * sumList t)        by S2
Q.E.D.
```